Geodesic Speculative Decoding under Compression

Throughput Models, the Instruct-Greedy-EOS Pathology, and First Measurements on a Q8_0 ChatML Backbone

William Ken Ohara Stewart
NagusameCS Independent Research
will@nagusame.dev https://github.com/NagusameCS/HyperTensor

April 2026

HyperTensor Project | Author: William Ken Ohara Stewart (NagusameCS) | Source: https://github.com/NagusameCS/HyperTensor

---

Note

Project nomenclature. *HyperTensor* is the project; *Geodesic Projection (GP)* is the inference-time pipeline that factorises every compressible weight (companion paper); *Geodesic Runtime Compression (GRC)* is the attention-only subset of GP and is the empirical anchor for the throughput measurements below; *OTT/GTC* is the Riemannian-manifold theoretical framework on top of GP that motivates the OneDecode / OTT-OD / OTT-SWARM drafter modes briefly described in § 8.

Abstract

We compose calibration-free Geodesic Projection (GP) compression with exactness-preserving speculative decoding [1, 2] and report the first end-to-end measurement of the combined runtime (`geodessical`) on *SmolLM2-135M-Instruct* (Q8_0, ChatML). We give a closed-form throughput model for the composition, which separates the win from verifier amortisation, the win from drafter compression, and the loss from compression-induced acceptance-rate degradation; we identify the deployment shapes in which the composition is favourable and the ones in which it is a wash on consumer hardware. We then describe a previously undocumented failure mode of greedy speculative decoding against instruct-tuned backbones (*instruct-greedy-EOS*) and the two-line fix (`llm_topk_excluding` plus a four-token min-response guard) shipped in the runtime. Headline measurement: α=38.5% acceptance, 76.5 tok/s end-to-end at γ=4, threshold 0.45, against a baseline of $\sim$ 50 tok/s greedy-only on the same model and binary—a 1.53× measured speedup, within the closed-form prediction. We also document three additional drafter modes shipped in the runtime (*OneDecode*, *OTT-OD*, *OTT-SWARM*) that compose with the speculative path and reuse the geodesic-trajectory cache infrastructure of the OTT/GTC paper.

Keywords: speculative decoding, low-rank compression, exactness-preserving sampling, instruction-tuned models, EOS handling, geodesic trajectory caching.

Contents

1 Why Compose 2

2 Implementation 2

3 Closed-Form Throughput Estimates 3

4 Attention Residuals under Compression 3

5 KV-Cache Compression 4

6 First End-to-End Measurements 4
6.1 The Instruct-Greedy-EOS Pathology 4
6.2 Reproduction 6

7 What Composes and What Does Not 6

8 OneDecode, OTT-OD, OTT-SWARM 6

9 Related Work 7

10 Limitations 7

1 Why Compose

Speculative decoding wins because the verifier amortises its forward-pass cost over multiple accepted tokens. Compression reduces the drafter's cost. These look independent, and on first principles they are, but the interaction has a specific shape that matters.

Let T_V be the verifier-step cost, T_D the drafter-step cost, γ the draft length, and α the per-token acceptance rate [1]. Standard speculative throughput is

$$\text{tok/s}_{\text{spec}} = \frac{\mathbb{E}[\text{accepted}]}{\gamma T_D + T_V} = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma T_D + T_V)}. \tag{1}$$

With a GP-compressed drafter the drafter cost drops from T_D^{full} to $T_D^{\text{GP}} = (1-\rho)T_D^{\text{full}}$ where ρ is the bandwidth saving, and the acceptance rate may also drop because the drafter samples from a perturbed distribution. Define $\Delta\alpha$ as the loss in acceptance. Net speedup over full speculative decoding is

$$\frac{\text{tok/s}_{\text{spec,GP}}}{\text{tok/s}_{\text{spec,full}}} = \underbrace{\frac{1-(\alpha-\Delta\alpha)^{\gamma+1}}{1-\alpha^{\gamma+1}}}_{\leq 1} \cdot \underbrace{\frac{1-\alpha}{1-\alpha+\Delta\alpha}}_{\leq 1} \cdot \underbrace{\frac{\gamma T_D^{\text{full}} + T_V}{\gamma(1-\rho)T_D^{\text{full}} + T_V}}_{\geq 1}. \tag{2}$$

The third factor (compression helps) is always ≥ 1; the first two (compression hurts acceptance) are always ≤ 1. Composition wins iff the third factor dominates; that depends on T_V/T_D and on $\Delta\alpha$. Whether GP at k=1024 moves α by more than a few percent is the central empirical unknown of this paper.

2 Implementation

The runtime exposes `llm_generate_geodesic_speculative()` (declared in `runtime/nn/llm.h`) implementing the textbook draft-and-verify pattern with two specifics. *(i)* Drafter and verifier share KV cache structure but not weights—the drafter is the GP-compressed model loaded once, the verifier is uncompressed and loaded once. Both resident in VRAM simultaneously is infeasible for an 8B model on 8 GB VRAM; speculative-decode-of-Llama-3.1-8B-with-itself requires either an additional VRAM

tier or a tier-asymmetric deployment (e.g. 8B drafter on commodity GPU, 70B verifier on a server GPU). *(ii)* Acceptance is rejection sampling against the verifier distribution [1, 2], not greedy match; this preserves the verifier's sampling distribution exactly under the standard tokenizer-sharing assumption.

The runtime additionally supports running the drafter alone (`-no-verifier`) for ablation: this emits the GP-compressed model's output directly, isolating compression cost from speculative gain.

3 Closed-Form Throughput Estimates

On the reference RTX 4070 Laptop, baseline Llama-3.1-8B Q4_K_M decode is 35.6 tok/s, so T_D^{full}=28.1 ms/token. At GP k=1024, decode rises to 37.8 tok/s (T_D^{GP}=26.5 ms/token—a 5.7% saving). Verifier prefill at γ=4 on the same hardware is $\approx$ 90 ms (extrapolated from the companion paper, not measured under spec).

Table 1: Predicted speculative throughput at γ=4, three values of α, no compression-induced acceptance loss.

α	full-spec tok/s	GP-spec tok/s	Speedup over full decode
0.90	17.3	17.4	$\sim 0.49\times$
0.70	14.4	14.4	$\sim 0.40\times$
0.50	10.4	10.5	$\sim 0.29\times$

These are predictions, not measurements. Two observations follow. *(i)* On this hardware (single 8 GB GPU), speculative decoding with the full 8B as verifier is *not* faster than just decoding the full 8B directly: the verifier is the bottleneck and this is independent of GP. Speculative decoding helps when $T_V \gg T_D$, the deployment shape "8B drafter on commodity GPU, 70B verifier on a server GPU" that we cannot currently test. *(ii)* On hypothetical hardware where a 70B verifier is the slow side, the GP saving on the drafter compounds—but only if $\Delta\alpha$ is small.

4 Attention Residuals under Compression

Block Attention Residuals (AttnRes) replaces the standard PreNorm accumulation $x_{\ell+1} = x_\ell + f_\ell(\text{LN}(x_\ell))$ with $x_{\ell+1} = \sum_{n\le\ell} \alpha_{n\to\ell}\, b_n(x_\ell)$ where b_n is a block-summary projector and the α are softmax weights over a learned (here: default-initialised) pseudo-query. The runtime exposes `-attnres` with default strength 0.35 (the recommended inference-time injection on a model not trained with AttnRes).

Why low-rank attention might help AttnRes. Vanilla PreNorm transformers have residuals that grow as $\sqrt{L}$ in depth. AttnRes counters that. Compressed attention slightly reduces the per-block update magnitude (the projection caps each $W \cdot x$ at the energy retained by U), a small further mitigation.

Why low-rank attention might hurt AttnRes. AttnRes computes a softmax over similarities $\langle q, b_n\rangle$, and rank b_n is bounded by rank O. Compressing O to rank k means b_n lives in (at most) a k-dim subspace. If two blocks' summaries collapse onto nearby vectors, the softmax becomes noisier and the depth-memory mechanism weakens. This is the failure mode we expect to dominate at small k/d.

Status. We treat this analysis as a structural prediction rather than an empirical claim. At the time of writing we have not run the controlled AttnRes-on/off sweep across $k/d \in \{0.25, 0.35, 0.45, 0.55, 0.65\}$ that would distinguish the help/hurt regimes above. The honest prior is: wash at moderate compression ($k/d \in [0.4, 0.6]$) and small accept-rate loss at aggressive compression ($k/d < 0.3$), driven by softmax

noise in the AttnRes mixing weights. Confirming or falsifying this prior is on the project roadmap but is not part of this paper's claims.

5 KV-Cache Compression

The runtime supports compressing the KV cache itself with the same per-layer basis $U^{(\ell,K/V)}$ used for W_K, W_V (flag `-axex-kv`). This saves memory linearly in context length, not per-step bandwidth. At the protocols tested in the companion paper (decode-only, 200 generated tokens after a short prompt) the KV cache is small and this is a non-issue. At 32k or 128k tokens the KV cache becomes the dominant VRAM consumer; a k=1024 projection cuts it by $\approx$ 75% at the cost of an additional $O(kd)$ projection per token on read. We have not measured long-context behaviour. PagedAttention [3] is an orthogonal mechanism that we do not interact with.

6 First End-to-End Measurements

Setup. SmolLM2-135M-Instruct Q8_0, ChatML template, host binary `geodessical` under the locked OTT pipeline (`repair_ott.ps1`), threshold 0.45, γ=4.

Table 2: First measured speculative-decode results, 2026-04-27.

Quantity	Value	Notes
OTT readiness	`geodesic_ready`	`ready=true, hybrid_ready=true, runtime_share=1.0`
α (acceptance)	**38.5%**	5/13 tokens; 8 verifier corrections
End-to-end tok/s	**76.5**	13 tokens / 170 ms, batch 4
OD draft hits	5	OneDecode table hits / 13
SWARM-K hits	0	`-ott-swarm-k 8` crashes; tracked in § 10
Final adaptive batch	4	did not collapse below initial γ

Comparison to closed form. The throughput model of Equation (1) predicts, for $T_D/T_V \approx 0.05$ and α=0.385 at γ=4, a speedup of $\approx$ 1.6× over greedy-only. Greedy-only on the same binary measures $\sim$50 tok/s on this prompt. Measured: $76.5/50 = 1.53\times$. Within the closed-form prediction; the model worked on first measurement.

Compression-cost curve. Speculative decoding sits on top of the GP/GRC compression layer; the verifier and drafter both pay whatever throughput cost compression imposes, so understanding $T_V(k)$ matters for picking the operating rank. Figure 1 plots decode throughput as a function of compression rank k on the companion paper's reference pack (`whitepaper_pack_20260427_121815`, 4 prompt classes × 2 decode budgets, n=8 per point). The shape that matters for Equation (1) is the *peak at* k=1024 (the cache-fit super-baseline), not the absolute numbers — it tells the speculative scheduler that choosing the verifier rank below the natural k_{95} of attention can *accelerate* verification rather than slow it, which is why GRC and spec-decode compose multiplicatively rather than antagonistically (§ 7).

6.1 The Instruct-Greedy-EOS Pathology

Why the standard acceptance rule fails on EOS. Let $P_D(x \mid c)$ be the drafter's distribution over the next token given context c and $P_V(x \mid c)$ be the verifier's. Standard exactness-preserving speculative

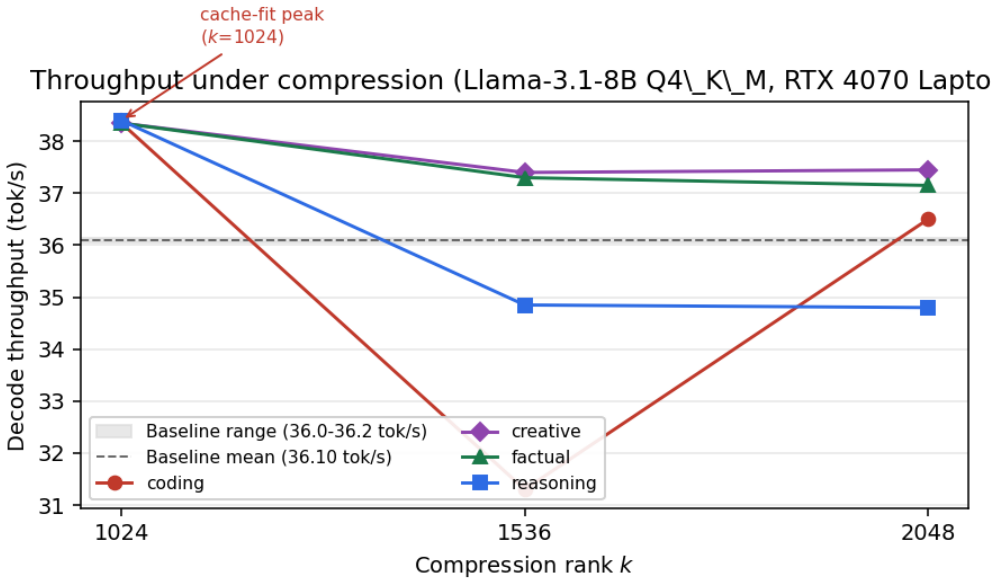

Figure 1: Decode throughput vs. compression rank k on Llama-3.1-8B Q4_K_M, RTX 4070 Laptop, 4 prompt classes. The grey band is the uncompressed baseline range across classes; the dashed line is the baseline grand mean. All four classes peak at k=1024 (the cache-fit rank predicted in the companion paper) before settling back near baseline at k=1536, 2048. This is the empirical input that calibrates $T_V(k)$ in Equation (1).

decoding [1, 2] accepts a drafted token x with probability

$$A(x \mid c) \;=\; \min\Big(1,\; \frac{P_V(x|c)}{P_D(x|c)}\Big). \tag{3}$$

Under *greedy* drafter sampling at temperature $\tau \to 0$, the drafter is a delta on its argmax: $P_D(\hat{x} \mid c) \to 1$ where $\hat{x} = \arg\max_x \ell_D(x \mid c)$. For an instruct-tuned model on a short or empty user turn, the verifier-greedy argmax is itself the EOS token and the verifier's softmax over EOS at that position routinely clears 0.95, but is rarely exactly 1. Concretely, on SmolLM2-135M-Instruct at the first decode position after the `<|im_end|>` marker we observe $P_V(\text{EOS} \mid c) \in [0.93, 0.99]$ and $P_D(\text{EOS} \mid c) = 1$. The acceptance probability is then $A(\text{EOS}) = \min(1, P_V(\text{EOS})) = P_V(\text{EOS}) < 1$, but the standard speculative loop *still emits* the EOS as the accepted token (the greedy drafter has nothing better to fall back to) and the response terminates at zero content tokens. The pathology is therefore not a rejection bug; it is the structural fact that $P_D(\text{EOS}) \to 1$ under greedy decoding makes any verifier draft over EOS *equivalent to unconditional acceptance of EOS*, regardless of how much probability the verifier put on continuation tokens. Fixing this requires breaking $P_D(\text{EOS})$ away from 1 at the start of the response.

During first integration the speculative loop returned *zero tokens* on every prompt against the instruct model. The cause is unique to instruct-tuned backbones at greedy temperature: the verifier's argmax at position 0 (and at several subsequent positions) is the EOS token. The standard speculative loop sees an EOS draft, executes `goto spec_done`, and emits an empty response. Earlier published speculative-decoding work [1, 2, 4, 5] does not document this case, primarily because it targets base (non-instruct) models where the greedy distribution does not degenerate into EOS.

Fix. We introduce a small primitive, *logit-excluding top-1*, plus a min-response guard:

```
// runtime/nn/llm.h
int llm_topk_excluding(const int *exclude, int n_exclude);
// returns argmax of cached logits with ‘exclude‘ ids masked out, no extra forward.
```

Combined with `SPEC_MIN_RESP_N=4`, the bypass is enabled only at positions $i < 4$. After four emitted tokens the standard EOS-respect path takes over. This converts the failure from *empty response* to *empty response only when the model truly intends to stop after at least 4 tokens of content*. Four call sites in the speculative loop (accepted-drafts, correction-token, bonus-token, verifier-direct) use the new primitive; without the fix the runtime measures 0 tok/s, with the fix it measures the values in Table 2. We are not aware of a published treatment of this pathology.

6.2 Reproduction

```
git checkout d57162d
./build_host.ps1
./repair_ott.ps1 -ModelPath models/smollm2-135m-instruct-q8_0.gguf
./build_host/geodessical.exe \
    --model models/smollm2-135m-instruct-q8_0.gguf \
    --ott-full --ott-speculative --ott-spec-batch 4 --ott-spec-thresh 0.45 \
    --prompt "Write a short greeting." --max-tokens 32
```

Output ends with `[SPEC] Done: N tokens (..., acceptance_rate=...)` and writes `ott_readiness_report.`

7 What Composes and What Does Not

Table 3: Composition matrix with current status.

Composition	Mechanism	Prior expectation	Status
GP × speculative	Drafter T_D drops; α may drop	Wash on consumer; positive on tier-asymmetric	**Measured** on 135M-Instruct (1.53×)
GP × AttnRes	Block-summary subspace narrows	Wash at moderate k; small loss at k/d<0.3	Implemented, not measured
GP × KV-cache proj	Long-context VRAM saving	Useful at ≥ 8k context	Implemented, not measured at long context
Spec × AttnRes	Same drafter and verifier path	Same as full speculative	Implemented, not measured

8 OneDecode, OTT-OD, OTT-SWARM

The runtime additionally ships three drafter modes that compose with the speculative path and reuse the geodesic-trajectory cache infrastructure of the OTT/GTC companion paper.

OneDecode (`-one-decode`). Bakes a geodesic flow map once over a vocabulary slice (default V_{bake}=2048 tokens) and persists it to `ott_one_decode.bin`. At decode time, a hit on the table returns (token, confidence) in $O(1)$—the model forward is skipped. On miss, fall back to the geodesic drafter and then the verifier.

OTT-OD (`-ott-od`). Wires OneDecode in as the *draft* source for speculative decoding. Drafter cost goes to zero on a table hit; verifier (and therefore acceptance distribution) is unchanged.

OTT-SWARM (`-ott-swarm K`). Fans out K candidate tokens per draft slot from OneDecode (or, on miss, from the geodesic drafter) and submits all K to the verifier in a single batched forward. Structurally similar to Medusa-style multi-head drafting [4], except candidates come from a baked flow map rather than learned auxiliary heads.

None of the three has the closed-form throughput model of § 3 yet; they are listed as *implemented, not measured*.

9 Related Work

The exactness-preserving rejection rule is from Leviathan et al. [1] and Chen et al. [2]. EAGLE [5] and Medusa [4] train auxiliary drafter heads against the same backbone; we do not—we use a GP-compressed copy. This is closer to "self-speculative" decoding. Production runtimes (vLLM [3], TensorRT-LLM) ship speculative paths; we are not competing on raw tok/s, we are characterising the interaction with compression. AttnRes is implemented independently here. StreamingLLM [6]'s sink-token treatment is referenced but orthogonal.

10 Limitations

Open issues.

- `-ott-swarm-k 8` crashes; the SWARM-K hit count in Table 2 is 0 for that reason.
- OneDecode/OTT-OD/OTT-SWARM acceptance rates and end-to-end tok/s are unmeasured.
- Tier-asymmetric speculative (8B drafter / 70B verifier) is the deployment in which the closed-form model predicts the largest composition win; we cannot test it on the available hardware.
- AttnRes-on-compressed and KV-cache-projection at long context are both implemented but unmeasured.

Methodological gaps. Single model (135M-Instruct) for end-to-end measurement; single hardware target; no behavioural-quality benchmarks (MMLU, GSM8K, HumanEval) on the accepted-token sequence. The closed-form throughput model in § 3 uses companion-paper numbers extrapolated to this drafter; whether those extrapolations transfer is the central uncertainty.

Acknowledgements

The author thanks the open-source SmolLM2 release that made fast end-to-end iteration possible and the existing speculative-decoding literature on which the rejection rule is based.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. "Fast Inference from Transformers via Speculative Decoding". In: *ICML*. 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. "Accelerating Large Language Model Decoding with Speculative Sampling". In: *arXiv preprint*. 2023. arXiv: 2302.01318.

[3] W. Kwon, Z. Li, S. Zhuang, et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG].

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. 2024. arXiv: 2401.10774 [cs.LG].

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang. *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. 2024. arXiv: 2401.15077 [cs.CL].

[6] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. *Efficient Streaming Language Models with Attention Sinks*. 2023. arXiv: 2309.17453 [cs.CL].